

Pretraining Data

pig7selene

September 5, 2026

Abstract

Large-scale pretraining data is not a passive collection of text. It is the result of a sequence of collection, extraction, filtering, deduplication, mixture, tokenization, and sequence-construction decisions that together define the distribution seen by a language model. This chapter develops those decisions at document, sample, sequence, and token levels. It explains how exact and near deduplication differ, why benchmark leakage compromises evaluation, how mixture weights change the learned distribution, and why valid-token budgets are a more useful scale measure than raw document counts.

1 Introduction

Chapter 5 defined pretraining as maximum-likelihood learning over token sequences. That formulation is deliberately compact: the loss averages the negative log-probability of observed target tokens. In practice, however, the observed tokens are produced by a data pipeline. Decisions about which documents enter the corpus, how they are cleaned, which duplicates are removed, how sources are sampled, and how text is packed into fixed-length sequences determine the empirical distribution on which the loss is estimated.

Pretraining data is therefore part of the model specification. A decoder-only Transformer trained on the same architecture and objective can learn materially different capabilities, biases, and failure modes when its input distribution changes. Modern open-corpus efforts make this point concrete by documenting mixtures that include filtered web text, scientific material, code, public-domain books, social media, and encyclopedic sources [1]. The objective of this chapter is not to prescribe one universal corpus. It is to establish the distinctions and contracts required to reason about a real language-model data pipeline.

Four granularities must remain separate. A **document** is a source artifact such as a web page, book chapter, repository file, or paper. A **sample** is a retained record after source-specific extraction and filtering; it may be one document or a selected span. A **sequence** is a fixed-length model input constructed from one or more samples. A **token** is the discrete unit on which the loss in Chapter 5 is evaluated. A decision at one level often changes another, but conflating the levels obscures both data provenance and the actual training distribution.

Unit	Typical representation	Typical decisions	Primary invariant
Document	Captured source artifact and metadata	Extraction, language assignment, provenance	Source identity remains recoverable.
Sample	Retained text record or span	Quality filters, deduplication, source weight	Filtering decision and reason are recorded.
Sequence	Fixed-length token-ID tensor	Packing, truncation, boundary policy	Inputs, labels, and masks are aligned.
Token	Vocabulary index	Loss eligibility and accounting	Counted under a declared tokenizer and mask.

Table 1: The four levels of a pretraining pipeline. A document is not automatically a training sequence, and a document count is not a token budget.

2 Data Sources

Large pretraining corpora commonly combine broad web collections with sources that have more explicit editorial, technical, or domain structure. Web data contributes scale and topical breadth, but it contains boilerplate, duplication, spam, malformed pages, and uneven language coverage. Books, reference works, scientific papers, code, forums, and licensed or public-domain collections can provide valuable distributions that are sparse in a generic crawl. The Pile illustrates a mixture-oriented design: it assembled 22 components, including academic and professional sources, rather than treating a single web corpus as sufficient [2].

A source category is not a quality label. A book may be scanned incorrectly; a code repository may contain generated files, secrets, or copied dependencies; a web page may contain unusually careful technical writing. Source selection should instead state a hypothesis about useful coverage, permitted use, and the processing that will be applied. A corpus inventory should retain source name, acquisition method, snapshot or release identifier, jurisdictional and license information when available, language coverage, and the intended sampling policy. These fields make later audits possible even when the raw source cannot be redistributed.

3 Data Collection and Crawling

Web-scale collection begins with a capture policy. A crawler or web archive records URLs, retrieval times, response metadata, and page payloads; a non-web source may instead provide a release manifest, file hashes, and version identifiers. The collection stage should preserve a stable document key independent of later cleaning. Otherwise, a pipeline cannot explain whether two retained samples came from the same raw artifact or whether a filter changed between reruns.

Collection is necessarily incomplete and selective. Crawl seeds, link-following rules, host restrictions, robots and access policies, retry behavior, time window, and archive availability all influence the captured distribution before any quality filter is applied. The Colossal Clean Crawled Corpus (C4), for example, begins from Common Crawl snapshots and applies a documented cleaning pipeline rather than claiming that a crawl is already a language-model corpus [3]. For every source, collection metadata and the raw-to-extracted mapping should be versioned alongside the model-facing data.

4 Text Extraction and Normalization

An HTML response is not a text document. Extraction removes markup, scripts, navigation menus, cookie notices, repeated templates, and other material that is useful for rendering a page but not necessarily for language modeling. The extraction rule must be evaluated on representative samples because aggressive boilerplate removal can erase tables, mathematical notation, code blocks, lists, or the main content of unusual page layouts. For structured sources, a parser should preserve distinctions that carry semantic meaning, such as paragraph boundaries, code indentation, document title, and section order, whenever the downstream representation can support them.

Normalization makes equivalent surface forms more consistent, but it is not a license to rewrite content. Common operations include Unicode canonicalization, removal of invalid control characters, repair of ill-formed byte sequences, and a declared treatment of whitespace. Replacing every newline with a space can destroy code and table structure; collapsing visually similar Unicode characters can alter identifiers; stripping all markup before extraction can concatenate unrelated navigation text. The appropriate output is a clean text record with a documented normalization version, not a vaguely defined string called “clean text.”

5 Language Identification

Language identification assigns a language label, a probability, or both to an extracted unit. It can operate at document, paragraph, or sentence level, and the choice matters. Document-level identification

is efficient and supports monolingual corpus construction, but it may discard legitimate multilingual documents or misclassify short, code-heavy, transliterated, and mixed-language pages. Finer-grained identification improves control but introduces boundary and aggregation decisions.

At web scale, language identification is a filter with a nonzero error rate, not an oracle. CCNet combines language identification, document deduplication, and quality selection to produce monolingual data from Common Crawl [4]. A production pipeline should retain classifier version, confidence score, threshold, and treatment of uncertain or mixed-language records. This permits a later analysis of what a label such as “English corpus” actually means and prevents an accidental threshold change from silently redefining the dataset.

6 Quality Filtering

Quality filtering aims to remove text that is unlikely to contribute useful learning signal under the intended objective. Typical candidates include empty extraction results, pages dominated by navigation or advertising, repeated character runs, machine-generated keyword lists, malformed encodings, extremely short fragments, and documents with implausible ratios of punctuation, digits, or stop words. Filters are often applied after extraction and language assignment, because their features are meaningful only on the retained textual representation.

No scalar quality score captures every useful property. A terse API reference, a mathematical proof, conversational dialogue, and literary prose can all look atypical under the same heuristic. Filters should be evaluated by inspecting both retained and rejected samples across source domains, languages, and length ranges. A conservative pipeline often separates clearly invalid material from uncertain material, records rejection reasons, and measures how each rule changes source composition. This makes it possible to distinguish a deliberate quality decision from an unintended collapse in diversity.

7 Heuristic and Model-Based Filtering

Heuristic filters use observable properties such as length, character composition, line structure, URL patterns, repetition, or a blacklist. They are fast, interpretable, and useful for removing obvious artifacts, but they encode a limited definition of quality. Model-based filters instead score a sample using a classifier, a language model, or similarity to a trusted reference distribution. CCNet uses perplexity-based filtering to select documents closer to high-quality corpora, showing one way in which a learned score can supplement hand-built rules [4].

Model-based filtering does not remove judgment; it relocates it into the training data, objective, calibration, and threshold of the scoring model. A classifier trained to recognize encyclopedic prose may remove useful informal dialogue or underrepresented dialects. Conversely, accepting every high-scoring sample can concentrate the corpus around a narrow style. RefinedWeb demonstrates that careful filtering and deduplication can yield a large high-quality web corpus [5], but the general lesson is not that one score is universally correct. It is that filter behavior must be measured against the target mixture and reviewed as a distributional intervention.

8 Deduplication

Duplication is introduced by crawl revisits, mirrors, syndicated pages, quoted material, templates, and repeated fragments within or across documents. It consumes token budget without adding independent evidence, can amplify narrow sources, and makes memorization more likely. Deduplication is performed on a declared representation: hashing raw bytes, normalized text, extracted paragraphs, or token sequences answers different questions. The representation, scope, and threshold must therefore be recorded with the data release.

Exact deduplication removes units that are identical under the selected canonicalization. A cryptographic hash of normalized document text is an efficient document-level test; exact substring methods can additionally find long repeated spans even when the surrounding documents differ. Exact matching has high precision but cannot recognize lightly edited copies, translation variants, or pages that differ only in a date, header, or advertising block.

Near deduplication identifies units that are sufficiently similar rather than identical. A common document-level approach converts a document into a set of token or character n -grams, uses MinHash or a related sketch to approximate Jaccard similarity, and applies locality-sensitive hashing to retrieve likely matches before an exact threshold check. The threshold controls a real tradeoff: a low threshold may remove independent documents that share a genre or template, whereas a high threshold leaves nearly identical copies. Lee et al. distinguish exact substring matching from approximate document matching and show that thorough deduplication can reduce memorized text and train-test overlap [6].

9 Contamination and Benchmark Leakage

Evaluation contamination occurs when material that reveals an evaluation item, its answer, or a close paraphrase appears in training data. The concern is not limited to an exact copy of a benchmark test question. Solutions, explanations, answer keys, mirrors, and derivative pages can permit recognition rather than the intended generalization. A reported benchmark score then mixes capability with exposure, and the magnitude of the error depends on both the overlap rule and the benchmark design. Contamination control should begin before training. A pipeline can maintain protected benchmark identifiers and canonicalized text, search candidate documents for exact or approximate overlap, and remove or quarantine matched records. The evaluation set itself should also be deduplicated internally and against the retained training set. The deduplication study of Lee et al. found train-validation overlap in standard datasets and emphasizes that it can distort evaluation [6]. Because no overlap detector is complete, a rigorous report states the protected benchmarks, matching representation, thresholds, scope, date of the data snapshot, and residual limitations rather than claiming contamination-free evaluation without evidence.

10 Data Mixtures and Sampling

After filtering and deduplication, a corpus is usually a family of source distributions rather than one homogeneous dataset. Let P_k denote the distribution induced by retained source component k , and let π_k be the probability that a training sequence is drawn from that component. The effective mixture is

$$P_{\text{mix}(x)} = \sum_{k=1}^K \pi_k P_{k(x)}, \quad \pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1. \quad (1)$$

Under the Causal Language Modeling objective of Chapter 5, expected training loss is taken with respect to P_{mix} , not an abstract distribution of all available text. Increasing π_k upweights the conditional patterns, vocabulary, styles, and domains represented by component k ; decreasing it does the opposite. Source size alone does not determine this effect. Sampling with replacement can repeat a small component many times, while a large component can be capped or downsampled. The Pile and Dolma both make source composition an explicit corpus-design choice rather than a by-product of raw storage volume [1], [2].

Mixture weights should be defined at the point where sampling occurs. If a pipeline first selects a document source, then packs documents into sequences, the intended document-level weights may differ from the realized token-level weights because documents have different lengths and rejection rates. Monitoring should therefore report both planned sampling probabilities and realized valid-token fractions after tokenization, truncation, packing, and loss masking.

11 Domain Balancing

Domain balancing is the deliberate choice to avoid allowing one abundant source to determine the entire mixture. Technical text, code, reference material, books, conversational text, and web pages each contain different conditional distributions. Upweighting a small component can preserve expertise or linguistic variety that would otherwise vanish in a web-dominated corpus. Downweighting can prevent repetitive, low-value, or legally sensitive sources from consuming a disproportionate share of the budget.

There is no source-independent optimal balance. A mixture should be evaluated against its intended use, language coverage, safety and governance requirements, and held-out distributions that are not themselves protected benchmark answers. Overweighting a polished subset can improve short-run loss while reducing stylistic and topical diversity. Conversely, preserving every noisy source at its raw frequency can waste training tokens. A useful practice is to make mixture changes as explicit experimental variables, with component-level ablations and token-accounting reports, instead of treating them as invisible preprocessing.

12 Tokenization and Sequence Construction

After document- and sample-level processing, retained text is converted to token IDs using the tokenizer described in Chapter 1. The tokenizer version, vocabulary, normalization configuration, and special-token policy must be pinned: changing any of them changes token counts, sequence boundaries, and the model’s input alphabet. Tokenization is therefore a model-facing transformation, not a storage detail that can be swapped after data mixture weights have been chosen.

Long samples must be divided, truncated, or streamed into model-length sequences. A deterministic prefix truncation rule can systematically underrepresent conclusions, appendices, and late-file code; random windows can improve coverage but must be seeded for reproducibility. Empty tokenizations, malformed special tokens, and documents that exceed a maximum length require explicit handling. At this stage, a sample can yield zero, one, or many sequences, so source proportions should be recomputed after the transformation rather than inferred from document counts.

13 Packing and Document Boundaries

Packing fills a fixed sequence length by concatenating tokenized material from multiple samples, often inserting an end-of-sequence marker between documents. It improves token utilization relative to padding every short sample to the context length. It also creates a boundary policy. If ordinary causal attention is used across the whole packed tensor, a token in a later document may attend to tokens from an earlier document. The end-of-sequence token signals a transition, but it does not mathematically prevent cross-document attention.

Some pipelines accept this behavior as an efficient approximation to a stream of documents. Others construct a block-diagonal attention mask or reset positional state at boundaries so that attention cannot cross documents. These choices change the conditional distribution being trained: the first trains on continuations conditioned on preceding packed material, while the second trains each document segment independently within the same tensor. The causal Attention Mask and loss mask have different roles, as Chapter 5 explains. Attention masking controls which prefix tokens are visible; a loss mask controls which target positions contribute to the scalar loss.

Padding, packing, and truncation must preserve target alignment. In a right-shift implementation, the final token of one packed unit can become the context for the first token of the next unless the boundary policy prevents it. Padding tokens should neither provide useful context nor contribute token-level loss. The data loader should make these facts observable through input IDs, target IDs, attention masks, loss masks, segment identifiers where used, and tests containing short hand-constructed examples.

14 Data Quality versus Data Quantity

Filtering aggressively can improve the average usefulness of a token, but it can also remove rare domains, minority language varieties, informal registers, or unconventional but valid technical text. Data quality is not simply the fraction of documents that resemble a preferred reference corpus. It is the suitability of a retained distribution for the intended learning problem, together with its provenance, diversity, duplication rate, and governance constraints.

Quantity still matters because a language model must observe broad lexical, factual, stylistic, and compositional variation. The relevant tradeoff is not a choice between “clean” and “large” data. It is an iterative allocation problem: estimate which filters remove artifacts, measure the diversity lost by each decision, inspect downstream behavior, and reserve enough unique material to avoid excessive repetition. RefinedWeb’s result that extensively filtered web data can remain large at scale is a useful counterexample to the claim that data quality and quantity are always opposed [5].

15 Dataset Scale and Token Budgets

Raw document count is a poor scale measure. Documents vary from a title fragment to a book, and their character counts do not map consistently to model work. Let $\tau(d)$ be the tokenization of retained document d . A corpus’s unique-token inventory can be summarized as

$$N_{\text{unique}} = \sum_{d \in \mathcal{D}} |\tau(d)|, \quad (2)$$

provided that the tokenizer and document-level deduplication policy are declared. Training scale is more directly described by the number of valid target tokens actually consumed. For sequences $s = 1, \dots, S$, with length L_s and loss mask $m_{s,t}$, define

$$N_{\text{drawn}} = \sum_{s=1}^S \sum_{t=1}^{L_s} m_{s,t}. \quad (3)$$

This quantity accounts for padding, excluded targets, sequence construction, and repeated sampling. It can exceed N_{unique} when the corpus is traversed for multiple epochs or when components are upsampled. Conversely, a large stored corpus may contribute fewer tokens than expected if filtering, truncation, or a source cap removes much of it. Compute-optimal scaling studies explicitly treat the number of training tokens as a variable coupled to model size and compute budget [7]. Any token-budget report should therefore name the tokenizer, whether counts include inputs or supervised targets, the mixture schedule, and the number of effective passes through each component.

16 Data Governance and Reproducibility

Data governance begins with provenance. Each retained sample should be traceable, as permitted by source policy, to a source component, capture or release version, extraction rule, language decision, filter decisions, deduplication cluster, and tokenizer version. A manifest can record content hashes rather than redistributing raw text when redistribution is restricted. This lineage supports removal requests, incident response, contamination investigation, and analysis of a model behavior that may be tied to a source component.

Reproducibility requires more than releasing a list of URLs. It requires deterministic or explicitly randomized pipeline stages, software and model versions, configuration files, thresholds, sampling seeds, corpus statistics before and after each filter, and immutable dataset manifests. Dolma’s release pairs a large corpus with curation details and tooling, illustrating the value of making construction decisions inspectable [1]. Some sources cannot be preserved indefinitely, so an honest data card should distinguish a reproducible procedure from an exactly reproducible byte-level snapshot.

17 Implementation Contracts

A data pipeline should expose a typed schema at every boundary. A raw document record needs a stable identifier, source metadata, capture metadata, and payload reference. An extracted sample needs normalized text, language information, quality scores or rejection reasons, deduplication state, and provenance links. A model sequence needs input IDs, target IDs, attention mask, loss mask, segment or document-boundary information where applicable, and a source-component identifier for accounting. Validation should run at every granularity. Document-level checks detect missing provenance and corrupt payloads. Sample-level checks measure acceptance rate, language distribution, quality-score distribution, and duplicate clusters by source. Sequence-level checks verify maximum length, special-token placement, packing policy, and the absence of invalid IDs. Token-level checks verify that valid loss positions contain targets in the vocabulary range and that the aggregate count agrees with Equation 3. A compact audit set of known duplicates, benchmark strings, multilingual pages, empty extractions, and boundary cases is more valuable than a pipeline that merely completes without throwing an exception.

18 Summary

Pretraining data is a sequence of distributional decisions rather than a static text archive. Collection and extraction define the document population; language identification, quality filtering, and deduplication decide which samples remain; mixture weights and domain balancing determine how often their patterns are observed. Exact and near deduplication answer different similarity questions, while contamination control protects evaluation from exposure to benchmark material and derivatives. Tokenization and sequence construction translate retained samples into the tensors used by the Causal Language Modeling objective. Packing, truncation, document boundaries, attention masks, and loss masks jointly determine which context is visible and which targets are counted. For this reason, a trustworthy pretraining corpus is described not by a document count alone but by source provenance, processing versions, mixture policy, unique-token inventory, and the valid-token budget actually drawn during training.

References

- [1] L. Soldaini *et al.*, “Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research,” *arXiv preprint arXiv:2402.00159*, 2024, [Online]. Available: <https://arxiv.org/abs/2402.00159>
- [2] L. Gao *et al.*, “The Pile: An 800GB Dataset of Diverse Text for Language Modeling,” *arXiv preprint arXiv:2101.00027*, 2021, [Online]. Available: <https://arxiv.org/abs/2101.00027>
- [3] C. Raffel *et al.*, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020, [Online]. Available: <https://jmlr.org/papers/v21/20-074.html>
- [4] G. Wenzek *et al.*, “CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data,” *arXiv preprint arXiv:1911.00359*, 2020, [Online]. Available: <https://arxiv.org/abs/1911.00359>
- [5] G. Penedo *et al.*, “The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only,” *arXiv preprint arXiv:2306.01116*, 2023, [Online]. Available: <https://arxiv.org/abs/2306.01116>
- [6] K. Lee *et al.*, “Deduplicating Training Data Makes Language Models Better,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2022, pp. 8424–8445. doi: 10.18653/v1/2022.acl-long.577.
- [7] J. Hoffmann *et al.*, “Training Compute-Optimal Large Language Models,” *arXiv preprint arXiv:2203.15556*, 2022, doi: 10.48550/arXiv.2203.15556.